



OS SECURITY

Introduction

- Security failures can cause major damage such as financial loss and harm.
- The operating system is the foundation of all software; if it is insecure, everything above it is too.
- The OS controls hardware resources like the CPU, memory, and devices, so a compromise means total control.
- Modern operating systems are large and complex, making them difficult to secure.
- The OS must protect processes and resources from untrusted programs.
- Applications rely on OS security guarantees such as file access control.
- If the OS fails, applications cannot achieve their own security goals.

What are we Protecting?

- The OS must protect everything on the system.
- A typical OS has full control over all hardware and can do anything the hardware allows.
- The OS can:
 - Access or modify any process's memory
 - Read, write, delete, or corrupt any file
 - Change scheduling or stop any process
 - Send or alter any message
 - Enable or disable any device
 - Give or remove any resource from a process
 - Respond falsely to system calls
- Processes are completely dependent on the OS for protection.
- Even a small flaw can let an attacker gain OS-level powers.
- Applying security patches and maintaining OS integrity is critical, since a compromised OS can expose the entire system.

Security Goals

- When we say a system is *secure*, we mean we can control what should and should not happen.
- We must define specific security goals before choosing defenses.
- Three main security goals:
 1. **Confidentiality:** Prevent unauthorized access to information.
Example: Keep credit card numbers hidden.
 2. **Integrity:** Prevent unauthorized changes to data or system state.
Example: Stop others from altering your files or transactions.
 3. **Availability:** Ensure systems and data remain usable for authorized users.
Example: Prevent attacks that block access to your services.
- These goals must support **controlled sharing:** who can access, modify, or use resources matters.

Security Policies and Enforcement

- **Non-repudiation:** Users should not be able to deny what they did.
Helps hold people accountable and discourages misuse.
- Security goals must be turned into **specific rules** the system can enforce.
Example: *Only users A and B can write to file X.*
- **Detailed policies** are needed because different users have different needs.
A system must allow sharing where needed but still block unwanted access.
- The **operating system provides tools** like permissions and access controls.
It can enforce policies, but it's up to administrators or users to set them correctly.
- Good security depends on both **strong mechanisms** and **clear, well-defined policies**.

Key Principles for Secure System Design

- **Economy of Mechanism:** Keep the system simple and small. Fewer bugs, easier to understand.
- **Fail-safe Defaults:** Default to secure settings, not insecure ones.
- **Complete Mediation:** Check every action against security policies every time.
- **Open Design:** Assume attackers know your system design; security should not rely on secrecy.
- **Separation of Privilege:** Require multiple credentials or parties for critical actions (e.g., 2FA).
- **Least Privilege:** Give users/processes only the permissions they need.
- **Least Common Mechanism:** Avoid sharing mechanisms or data structures across users/processes.
- **Acceptability:** Security must be usable; if users avoid it, the system fails.

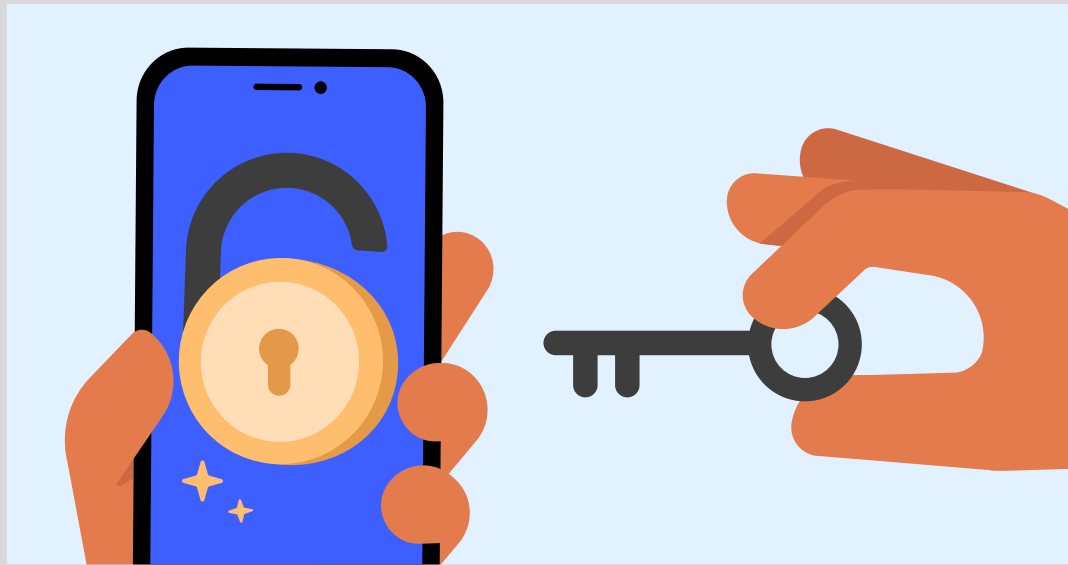
OS Security Basics: Goals & Built-in Protections

- Security goals focus on **confidentiality, integrity, availability**.
- Some goals are **built-in**, others are **user-defined**.
- Built-in goals often control **process access to hardware**.
- OS services (file system, memory, IPC) must be **carefully controlled**.
- **Clean separation of processes** is crucial:
 - Prevents one process from interfering with another.
 - Requires careful management of hardware and services.
- **Virtual memory** helps enforce process isolation:
 - Each process can only access its mapped memory.
 - Hardware prevents unauthorized memory access.

OS Security: System Calls & Access Control

- Processes request OS services via **system calls**.
- System calls switch from **user mode** → **supervisor mode**.
- OS checks:
 - **Which process made the call**
 - **Which service is requested**
- OS enforces **security policies**:
 - Authorized → service allowed
 - Unauthorized → error returned
- Includes control over **peripheral devices** via drivers.
- **Process Control Block (PCB)** used for **access checks and authorization**.

Authentication



OS Security: Who's Asking? Why Context Matters

- The OS provides services, but not everything should be allowed.
- Key question: **Who is asking for the service?**
 - Example: Admin asks to install a program → usually OK
 - Random script asks → be cautious!
- These “who” entities are called **principals** (users, groups, or programs).
- Knowing **who is asking** is essential for security decisions.

How the OS Controls Access

- A process acts as an **agent** for a principal.
- The thing being accessed is called an **object** (file, memory, device).
- The OS checks if the process is allowed to access it.
- Keeps track of permissions with **credentials** (like page tables for memory).
- If there's no existing permission, the OS checks the principal's identity.

Getting Identity Right

- Processes need the correct identity to enforce policies.
- Types of identity:
 - User (person)
 - Group (team)
 - Program (software)
- Mistakes can be serious:
 - Wrong identity → someone gets access they shouldn't
 - Or legitimate access gets blocked
- Once a process is authenticated, the system **trusts it for its lifetime.**
- Main challenge: **How do we securely assign and maintain process identities?**

How Processes Get Their Identity

- New processes are usually **created by other processes** (parent → child).
- Simple method: **child inherits parent's identity**.
 - OS copies identity from parent's process control block (PCB) to child's PCB.
- Single identity works for a basic system, but **won't support multi-user policies**.
- **Multi-user systems:**
 - Assign identities based on the **human user** who owns the process.
 - Example:
 - Shell process (command-line)
 - Window manager process (graphical system)
- When you start a program, the OS creates a new process **under your identity**.
- OS must **verify user identity at login** before attaching it to processes.
- This ensures reliable identities for enforcing **security policies**.



What you know –
passwords, PINs; example:
Roman “watchwords”



What you have – security
tokens, ID cards:
example: Roman letter of
recommendation



What you are –
biometrics; example:
Biblical “shibolleth”



What you know

What you have

What you are

Authentication by What You Know: Passwords

- A password is a secret **only you know**- you prove your identity to the system.
- The system doesn't need to "know" your password itself.
- **Password Hashing**
 - Store a cryptographic hash of the password, not the password itself.
 - When logging in, the system hashes the user's input and compares it to the stored hash.
- **Salting**
 - Add a random value (salt) to each password before hashing.
 - Prevents attackers from precomputing hash dictionaries.
- **Strong Passwords**
 - Long, mixed-character passwords resist guessing.
 - Avoid dictionary words or predictable sequences.
- Use **password vaults** to manage multiple strong passwords.
- Combine passwords with other factors (**multi-factor authentication**) for better security.

Authentication by What You Have

- **Everyday examples:** ID cards, Event tickets, ATM cards
- **Computer examples:**
 - **Hardware tokens / dongles** – plug into USB to verify you have the device
 - **Smart tokens** – display a changing code; user types it in for authentication
- Codes often **change frequently** to prevent theft from turning into “something you know”
- Losing your device = problem: can’t log in, and someone else could misuse it
- **Multi-factor authentication** helps: even if device is stolen, password is still required
- **Real-world considerations:**
 - Text-message codes (SMS) aren’t perfect, but usually secure enough in practice
 - Convenience vs. strict security: perfect security often isn’t practical
- **Takeaway:**

Authentication by what you have works well but always consider backup plans and combining methods for stronger security.

Authentication by What You Are (Biometrics)

- Uses **unique human traits** (physical or behavioral) to identify users: fingerprints, face, retina, typing patterns.
- **Challenges:**
 - Human features change (lighting, angle, hairstyle, masks).
 - Computers process features as numbers; exact matches are rare → need **feature-based comparison**.
- **Errors:**
 - **False positive:** imposter accepted.
 - **False negative:** real user rejected.
 - Tradeoff exists; often measured by **Crossover Error Rate (CER)**.
- **Practical issues:**
 - Special hardware may be needed.
 - Remote biometric data can be spoofed.
- No method is perfect; combine biometrics with other factors (multi-factor authentication) for reliability and security.

Authenticating Non-Humans

- **What are non-human users?**
 - Processes not associated with a human (e.g., web servers, smart devices).
 - These processes still need identities in the OS (like `webserver` or `lightbulb`).
- **How to authenticate non-human processes:**
 - Assign passwords to non-human users (used when creating processes).
 - Admins can create processes for these users directly or temporarily change process ownership.
 - Example: `sudo -u webserver apache2` → starts process as `webserver`.
- **Role of system administrators:**
 - Trusted admins can assign identities without needing the non-human user's credentials.
 - Ensures only authorized processes can act under sensitive identities.
- **Groups for access control:**
 - Multiple users can share privileges via groups.
 - Example: Several admins can start a web server by belonging to a `webadmins` group.
 - Groups provide flexibility in managing permissions and security policies.
- Non-human authentication ensures system integrity and controlled access to resources.
- Both individual and group identities are important in modern OS security.

Access Control

Introduction to Access Control

- Turn security policies into actionable decisions for the system.
- **Two key steps:**
 - **Policy Check:** Determine if the request fits within security policies.
 - **Enforce Decision**
 - Allow the operation if permitted.
 - Deny the operation if forbidden.
- **Example:**
 - User X wants to read/write `/var/foo`.
 - Process issues a system call: `open("/var/foo", O_RDWR)`
 - OS must decide: **Allow or Deny?**

Key Concepts in Access Control

- **Access Control Basics:**

- **Subject:** Who wants access (user or process)
- **Object:** What is being accessed (file, device, etc.)
- **Access:** How the object is used (read, write, execute)
- **Authorization:** The process of deciding if access is allowed

- **Reference Monitor:**

- Software that enforces access control rules
- Must be **correct** (security) and **efficient** (performance)
- Balances thorough security checks with system speed

- **Efficiency Methods:**

- Give subjects “virtual objects” that are theirs alone (e.g., virtual memory)
- OS ensures these objects are protected behind the scenes
- Reduces need for repeated checks while maintaining security

Approaches to Access Control

- **Two Main Approaches:**

- 1. Access Control Lists (ACLs)** – Like a guest list:

- The system checks if the subject is allowed access to the object
 - Example: Is user X allowed to open /tmp/foo?

- 2. Capabilities** – Like a key:

- The subject presents a token/capability to gain access
 - Example: Process holds a capability for /tmp/foo

Capabilities for Access Control: The Concept

- **Capabilities = Keys or Tickets**

- Each process has a set of capabilities specifying what it can access
- Unlike ACLs, capabilities encode the **entire permission set** for a process
- OS checks the capability on each access request (e.g., `open("/tmp/foo")`)

- **Security Considerations:**

- Capabilities are just bits → can be copied or forged if unmanaged
- OS keeps capabilities in **protected memory**
 - Processes request access through system calls
 - OS enforces policy and prevents unauthorized transfer
- Supports fine-grained privilege control for child processes

- **Historical Examples:** Hydra OS, early hardware/software systems (Dennis & van Horn, Wulf et al.)

Advantages & Practical Use

- **Advantages of Capabilities:**

- Easy to determine what a process can access (check its capability list)
- Easy to grant/revoke limited privileges to processes
- Can be efficient: capability can point directly to resource, reducing repeated checks

- **Challenges:**

- Harder to see all users who can access a resource (unlike ACLs)
- Must prevent forgery and manage storage efficiently

- **Practical Use in Modern OS:**

- Most OS use **ACLs for user-visible access control**
- **Capabilities under the hood** improve efficiency:
 - Example: Linux `open()`
 - ACL checked once on open
 - A capability-like structure attached to the process PCB for subsequent reads/writes
 - Reduces repeated ACL checks and improves performance

Mandatory vs Discretionary Access Control (MAC vs DAC)

- Who decides who can access a resource?
- **Discretionary Access Control (DAC):**
 - Resource owner decides access.
 - Example: A user sets permissions on their own files.
 - Flexible, but security depends on the owner making the right decisions.
- **Mandatory Access Control (MAC):**
 - Authority (e.g., system administrator or security officer) sets rules.
 - Users cannot override these rules.
 - Example: Military Top Secret information: even if you create a document, you can't give access to someone without proper clearance.
- Often combined with discretionary restrictions: users can further restrict access but **cannot relax mandatory rules.**
- DAC vs MAC is independent of **ACLs vs capabilities.**
- MAC systems involve the OS to enforce policies strictly.
 - Rarely seen in general computing, but critical in high-security environments.

Practical Access Control: Defaults, Roles, and RBAC

Defaults and Usability:

- Users rarely tweak access permissions manually.
- Systems provide default ACLs (e.g., `umask` in Linux) for new files.
- Proper defaults are critical to maintain security without user intervention.

Role-Based Access Control (RBAC):

- Organizes access by roles instead of individual users.
- Users gain permissions by assuming a role; lose them when leaving it.
- Supports multiple, disjoint roles per user.
- Example: A programmer can switch to a manager role temporarily, restricting access to code libraries or sensitive reviews.
- Enhances **least privilege**: users only get permissions necessary for their current role.

Practical Implementations:

- Minimal RBAC via ACLs, groups, and `setuid/sudo`.
- `sudo -u Programmer install newprogram` allows running commands under a role safely.
- Full RBAC systems allow finer-grained control and better enforcement.
- Often combined with **mandatory access control** to prevent role misuse.
- **Security Principle:**
 - Least privilege + role enforcement prevents accidental or malicious access.



Thank You